Bài 2.6 – Bias–Variance, Overfitting/Underfitting, Regularization L1/L2

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 2 buổi · Hình thức mở đầu: *Mô phỏng phỏng vấn*

🎯 Sau bài này bạn sẽ

- Nhận ra **underfitting** (học quá ít) và **overfitting** (học vẹt) bằng cách so sánh lỗi train với lỗi test.
- Hiểu đánh đổi **Bias – Variance**, khái niệm quan trọng nhất để chẩn đoán mô hình.
- Dùng **Ridge (L2)** và **Lasso (L1)** để "kìm" mô hình học vẹt, chọn được mức `alpha`.
- Biết Lasso còn giúp **tự chọn đặc trưng**, Ridge giúp ổn định hệ số khi các cột trùng thông tin.

Chia buổi:

- **Buổi 1:** Mục 1 → 4 (underfit, overfit, bias–variance, cách chữa).
- **Buổi 2:** Mục 5 → 9 (Ridge, Lasso, áp dụng cho giá nhà, bài tập).

---

1. Mô phỏng phỏng vấn 🎤

> **Người phỏng vấn:** "Mô hình của em đạt R^2 = 0,99 trên tập train nhưng chỉ 0,40 trên tập test. Chuyện gì đang xảy ra? Em xử lý thế nào?"

> **Người phỏng vấn:** "Còn nếu cả train và test đều chỉ đạt 0,55 thì sao?"

> **Người phỏng vấn:** "Ridge và Lasso khác nhau thế nào? Khi nào em chọn Lasso?"

Ba câu này xuất hiện trong hầu hết các buổi phỏng vấn ML. Tự trả lời trước. Gợi ý ở mục 8.

---

2. Thí nghiệm: giá xe máy cũ

2.1. Dữ liệu

Ta **mô phỏng** dữ liệu giá xe máy cũ (triệu đồng) theo số năm sử dụng. Quy luật thật (ta biết vì tự tạo ra): giá giảm nhanh những năm đầu, chậm dần về sau, cộng thêm nhiễu ngẫu nhiên (xe giữ gìn tốt hay xấu...).

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.pipeline import make_pipeline
from sklearn.metrics import root_mean_squared_error

rng = np.random.default_rng(1)
def tao_du_lieu(n):
    nam = rng.uniform(0, 10, n)                                  # số năm sử dụng: 0–
10
```

```
    gia = 30 * np.exp(-0.25 * nam) + rng.normal(0, 1.5, n)    # quy luật thật +
nhiễu
    return nam.reshape(-1, 1), gia

X_train, y_train = tao_du_lieu(20)      # chỉ 20 xe để học
X_test, y_test = tao_du_lieu(200)       # 200 xe để kiểm tra
```

2.2. Thêm "độ cong" cho Linear Regression

Đường thẳng không uốn cong được. Mẹo: tạo thêm các cột x^2, x^3, ..., x^k rồi đưa vào Linear Regression. Mô hình vẫn "tuyến tính theo hệ số", nhưng đường vẽ ra thì **cong**. Gọi là **Polynomial Regression (Hồi quy đa thức)**, và k gọi là **bậc**.

```
 x = 2  ──►  PolynomialFeatures(3)  ──►  [1, 2, 4, 8]   (1, x, x², x³)
```

`make_pipeline` nối các bước lại thành một mô hình duy nhất (sẽ học kỹ ở bài 2.17).

2.3. Ba mô hình, ba mức độ phức tạp

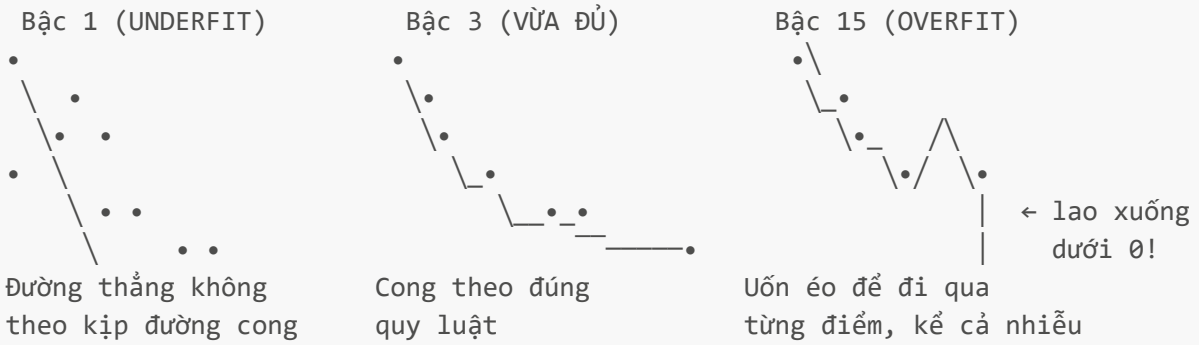
```
import matplotlib.pyplot as plt

x_ve = np.linspace(0, 10, 300).reshape(-1, 1)        # các điểm để vẽ đường cong
mượt
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, bac in zip(axes, [1, 3, 15]):
    mh = make_pipeline(PolynomialFeatures(bac), StandardScaler(),
LinearRegression())
    mh.fit(X_train, y_train)
    tr = root_mean_squared_error(y_train, mh.predict(X_train))
    te = root_mean_squared_error(y_test, mh.predict(X_test))
    print(f"Bậc {bac:>2}: RMSE train = {tr:5.2f} | RMSE test = {te:8.2f}")
    ax.scatter(X_train, y_train)
    ax.plot(x_ve, mh.predict(x_ve), "r")
    ax.set_ylim(-5, 40); ax.set_title(f"Bậc {bac}")
plt.show()
```

Kết quả mẫu:

```
Bậc  1: RMSE train =  3.21 | RMSE test =    2.70
Bậc  3: RMSE train =  1.50 | RMSE test =    1.71
Bậc 15: RMSE train =  1.18 | RMSE test =   15.95
```

Bạn sẽ thấy:



Mô hình	Lỗi train	Lỗi test	Chẩn đoán
Bậc 1	3,21 (cao)	2,70 (cao)	**Underfitting**: quá đơn giản
Bậc 3	1,50	1,71	**Vừa đủ**
Bậc 15	**1,18 (thấp nhất)**	**15,95 (tệ nhất!)**	**Overfitting**: học vẹt

Mô hình bậc 15 có lỗi train **thấp nhất** nhưng lỗi test **gấp gần 10 lần** mô hình bậc 3. Ở khoảng 9–10 năm, đường cong của nó vọt lên rồi lao xuống dưới 0: dự đoán giá xe **âm**!

---

3. Khái niệm

3.1. Underfitting (Học chưa tới)

- **Định nghĩa:** Mô hình quá đơn giản, không nắm được quy luật thật. Sai nhiều **cả trên train lẫn test**.
- **Ví dụ đời thực:** Học sinh chỉ học thuộc "đáp án thường là C". Làm bài tập hay đề thi đều điểm thấp.

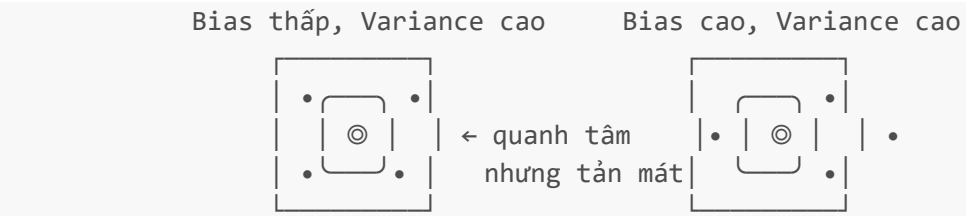
3.2. Overfitting (Học vẹt)

- **Định nghĩa:** Mô hình quá phức tạp, học thuộc cả **nhiễu ngẫu nhiên** trong dữ liệu train. Rất tốt trên train, **kém trên dữ liệu mới**.
- **Ví dụ đời thực:** Học sinh học thuộc lòng từng chữ lời giải trong sách. Gặp đúng bài trong sách thì 10 điểm, đổi số một chút là không làm được.

3.3. Bias và Variance

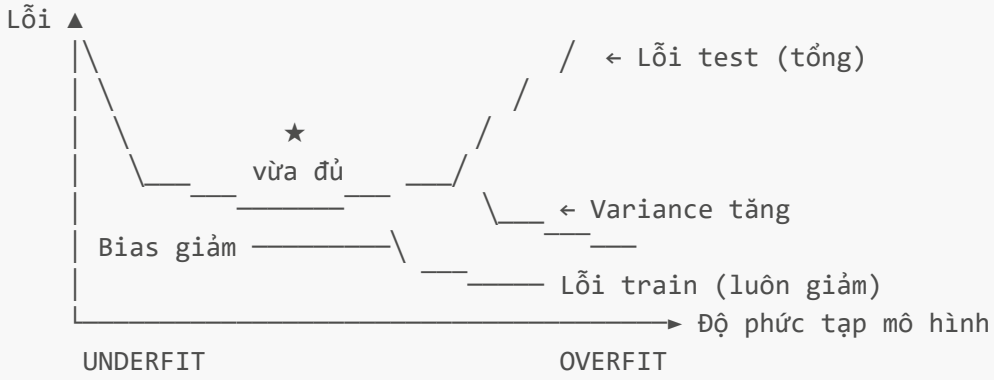
- **Bias (Độ chệch):** Lỗi do mô hình **quá cứng nhắc**, có "định kiến" sai về hình dạng quy luật. Bậc 1 có bias cao: nó **luôn** nghĩ quy luật là đường thẳng.
- **Variance (Phương sai):** Lỗi do mô hình **quá nhạy** với dữ liệu train cụ thể. Đổi 20 xe khác, mô hình bậc 15 sẽ vẽ ra một đường cong **hoàn toàn khác**. Variance cao.





(Mỗi • là một lần huấn luyện trên một bộ dữ liệu train khác nhau; ◎ là đáp án đúng)

3.4. Đánh đổi Bias–Variance



- Tăng độ phức tạp → bias giảm, variance tăng.
- Lỗi **train luôn giảm** khi mô hình phức tạp hơn. Vì vậy **không bao giờ** chọn mô hình dựa trên lỗi train.
- Điểm tốt nhất (★) là nơi **lỗi test/validation** thấp nhất.

3.5. Bảng chẩn đoán nhanh

Lỗi train	Lỗi val/test	Chẩn đoán	Cách chữa
Cao	Cao (≈ train)	**Underfitting**	Mô hình phức tạp hơn; thêm đặc trưng tốt; giảm regularization
Thấp	**Cao hơn nhiều**	**Overfitting**	Thêm dữ liệu; mô hình đơn giản hơn; **regularization**; bớt đặc trưng
Thấp	Thấp (≈ train)	✅ Tốt	Kiểm tra rò rỉ dữ liệu cho chắc

Mô hình giá nhà ở bài 2.3 (R^2 train 0,55 ≈ test 0,54): **underfitting**.

---

4. Cách chữa overfitting số 1: thêm dữ liệu

```
X_nhieu, y_nhieu = tao_du_lieu(500)       # 500 xe thay vì 20
mh = make_pipeline(PolynomialFeatures(15), StandardScaler(),
LinearRegression()).fit(X_nhieu, y_nhieu)
```

```
print(round(root_mean_squared_error(y_nhieu, mh.predict(X_nhieu)), 2),
      round(root_mean_squared_error(y_test, mh.predict(X_test)), 2))
```

Kết quả mẫu:

```
1.54 1.4
```

Cùng mô hình bậc 15, nhưng với 500 điểm, lỗi test giảm từ 15,95 xuống **1,4**. Có quá nhiều điểm "ghim" đường cong lại, nó không thể uốn éo tùy ý được nữa.

Nhưng dữ liệu thường **đắt** hoặc **không có thêm**. Khi đó ta dùng regularization.

---

5. Regularization (Chính quy hóa)

5.1. Trực giác

Nhìn vào hệ số của mô hình bậc 15:

```
mh0 = make_pipeline(PolynomialFeatures(15), StandardScaler(),
LinearRegression()).fit(X_train, y_train)
print("Tổng |w| của Linear Regression bậc 15:", round(np.abs(mh0[-1].coef_).sum(),
1))   # mh0[-1]: bước cuối của pipeline
```

Kết quả mẫu:

```
Tổng |w| của Linear Regression bậc 15: 135355.1
```

Hệ số **khổng lồ** (hàng trăm nghìn), dương âm triệt tiêu nhau để đường cong đi qua đúng từng điểm. Đó là dấu hiệu của học vẹt.

Ý tưởng regularization: Thêm vào loss một khoản **phạt hệ số lớn**. Mô hình phải cân bằng giữa "khớp dữ liệu" và "giữ hệ số nhỏ". Hệ số nhỏ → đường cong **mượt hơn**, ít uốn éo.

- **Ví dụ đời thực:** Giới hạn ngân sách khi đi chợ. Không giới hạn, bạn mua đủ thứ vô dụng. Có giới hạn, bạn chỉ mua thứ thật sự cần.

5.2. Ridge Regression (L2)

$$Loss_Ridge = MSE + \alpha \times (w_1^2 + w_2^2 + \ldots + w_n^2)$$

Ký hiệu	Ý nghĩa
MSE	Lỗi khớp dữ liệu như cũ
$w_1^2 + ... + w_n^2$	Tổng bình phương hệ số (gọi là **chuẩn L2**). Không phạt b
α (alpha)	**Mức phạt** do bạn chọn. α = 0: như Linear Regression thường. α lớn: hệ số bị ép về gần 0

Tính thử: hai mô hình cùng MSE = 2. Mô hình A có w = [10, −8], mô hình B có w = [1, 0,5]. Với α = 0,1:

- A: 2 + 0,1 × (100 + 64) = 2 + 16,4 = **18,4**
- B: 2 + 0,1 × (1 + 0,25) = 2 + 0,125 = **2,125**

Ridge chọn B: khớp dữ liệu y như nhau, nhưng "đơn giản" hơn.

```
for alpha in [0.001, 0.1, 1, 10, 100]:
    mh = make_pipeline(PolynomialFeatures(15), StandardScaler(),
Ridge(alpha=alpha))
    mh.fit(X_train, y_train)
    w = mh[-1].coef_
    print(f"alpha = {alpha:>6}: RMSE train = {root_mean_squared_error(y_train,
mh.predict(X_train)):5.2f}"
          f" | test = {root_mean_squared_error(y_test, mh.predict(X_test)):5.2f} |
tổng |w| = {np.abs(w).sum():8.1f}")
```

Kết quả mẫu:

```
alpha =  0.001: RMSE train =  1.29 | test =  3.37 | tổng |w| =    120.4
alpha =    0.1: RMSE train =  1.58 | test =  1.78 | tổng |w| =     34.0
alpha =      1: RMSE train =  2.28 | test =  2.05 | tổng |w| =     19.0
alpha =     10: RMSE train =  3.80 | test =  3.66 | tổng |w| =      5.7
alpha =    100: RMSE train =  5.17 | test =  5.69 | tổng |w| =      3.0
```

Đọc:

- Chỉ cần α = 0,001, tổng hệ số giảm từ 135.355 xuống 120, lỗi test giảm từ 15,95 xuống 3,37.
- α = **0,1** cho lỗi test tốt nhất (1,78), gần bằng mô hình bậc 3 "vừa đủ".
- α quá lớn (100) → hệ số bị ép quá mạnh → **underfitting** (train và test đều tệ).

α điều khiển vị trí trên đường cong bias–variance. α nhỏ: nghiêng về overfit. α lớn: nghiêng về underfit. Chọn α bằng **validation** (bài 2.18 sẽ tự động hóa việc này).

5.3. Lasso Regression (L1)

$$Loss_{Lasso} = MSE + \alpha \times (|w_1| + |w_2| + ... + |w_n|)$$

Khác Ridge ở chỗ phạt **trị tuyệt đối** thay vì bình phương. Hệ quả bất ngờ: Lasso đẩy nhiều hệ số về **đúng bằng 0**.

```
for alpha in [0.001, 0.01, 0.1, 1]:
    mh = make_pipeline(PolynomialFeatures(15), StandardScaler(),
Lasso(alpha=alpha, max_iter=100000))
    mh.fit(X_train, y_train)
    w = mh[-1].coef_
    print(f"alpha = {alpha:>5}: RMSE test = {root_mean_squared_error(y_test,
mh.predict(X_test)):5.2f}"
          f" | số hệ số khác 0 = {(np.abs(w) > 1e-6).sum()}/{len(w)}")
```

Kết quả mẫu:

```
alpha = 0.001: RMSE test =  3.04 | số hệ số khác 0 = 6/16
alpha =  0.01: RMSE test =  1.78 | số hệ số khác 0 = 5/16
alpha =   0.1: RMSE test =  1.64 | số hệ số khác 0 = 3/16
alpha =     1: RMSE test =  3.17 | số hệ số khác 0 = 1/16
```

Với α = 0,1, Lasso **chỉ giữ 3 trong 16** đặc trưng và đạt lỗi test tốt nhất (1,64), tốt hơn cả mô hình bậc 3. Nó **tự chọn đặc trưng** cho bạn.

Vì sao L1 tạo ra số 0 mà L2 thì không? Với L2, khi w đã nhỏ (ví dụ 0,01), khoản phạt w^2 = 0,0001 gần như không còn, nên không có lực đẩy tiếp về 0. Với L1, khoản phạt |w| giảm **đều đặn** đến tận 0, nên luôn có lực đẩy về 0 với cùng cường độ.

5.4. So sánh

	Ridge (L2)	**Lasso (L1)**
Phạt	Tổng w^2	Tổng |w|
Hệ số	Nhỏ đi, **hiếm khi bằng 0**	Nhiều hệ số **bằng đúng 0**
Tự chọn đặc trưng	Không	**Có**
Đặc trưng tương quan mạnh	Chia đều trọng số, **ổn định**	Hay chọn ngẫu nhiên 1 cột, bỏ các cột kia
Dùng khi	Nhiều đặc trưng đều có ích một chút	Nghi nhiều đặc trưng vô dụng; cần mô hình gọn

Muốn cả hai: **ElasticNet** (kết hợp L1 + L2).

> ⚠️ **Luôn chuẩn hóa** trước Ridge/Lasso. Khoản phạt tính trên độ lớn hệ số, mà độ lớn hệ số phụ thuộc đơn vị của cột (bài 2.2). Không chuẩn hóa = phạt không công bằng giữa các cột.

> 💡 `LogisticRegression` của Scikit-learn **mặc định đã có L2**. Tham số `C` = 1/α: **C càng nhỏ, phạt càng mạnh** (ngược với alpha!).

6. Ridge chữa "đa cộng tuyến"

- **Multicollinearity (Đa cộng tuyến):** Hiện tượng nhiều đặc trưng tương quan rất mạnh với nhau (nói cùng một điều). Hệ số Linear Regression trở nên **không ổn định**.

Các cột "tổng" của dữ liệu nhà tương quan rất mạnh (0,86–0,98). Lấy 3 mẫu ngẫu nhiên 300 khu, huấn luyện lại:

```
df = nap_du_lieu_nha()              # ô chuẩn bị dữ liệu ở bài 2.1
cot = ["tong_phong", "tong_phong_ngu", "dan_so", "so_ho", "thu_nhap"]
for seed in [1, 2, 3]:
    mau = df.sample(300, random_state=seed)                # lấy ngẫu nhiên 300
khu
    Xm = StandardScaler().fit_transform(mau[cot])
    lr = LinearRegression().fit(Xm, mau["gia_nha"])
    rd = Ridge(alpha=10).fit(Xm, mau["gia_nha"])
    print(seed, "LR:", lr.coef_.round(1), "| Ridge:", rd.coef_.round(1))
```

Kết quả mẫu:

```
1 LR: [-56.1  27.9 -32.1  57.5  68.9] | Ridge: [-35.1  18.7 -25.1  40.5  62.7]
2 LR: [-82.3  54.3 -57.7  73.6  78.4] | Ridge: [-41.9  30.5 -39.2  41.4  68. ]
3 LR: [-28.   -8.  -50.1  83.   71.2] | Ridge: [-21.7  17.3 -35.4  37.5  67.7]
```

Hệ số `tong_phong_ngu` (cột 2) của Linear Regression nhảy từ +27,9 lên +54,3 rồi **đổi dấu** thành −8,0. Không thể giải thích "phòng ngủ làm tăng hay giảm giá" được! Ridge dao động ít hơn hẳn (18,7; 30,5; 17,3) và giữ đúng dấu. Hệ số `thu_nhap` (cột 5, không tương quan với các cột khác) thì ổn định ở cả hai.

7. Áp dụng: giá nhà California

Bài 2.3 chẩn đoán mô hình bị **underfitting**. Cách chữa: tăng độ phức tạp bằng đặc trưng đa thức bậc 3 (8 cột → 165 cột, gồm mọi tích như thu_nhap × km_tp_lon, thu_nhap²...). Nhưng 165 cột dễ overfit, nên cần regularization.

```
from sklearn.model_selection import train_test_split

cot8 = ["thu_nhap", "km_tp_lon", "phong_moi_ho", "ti_le_phong_ngu",
        "nguoi_moi_ho", "tuoi_nha", "kinh_do", "vi_do"]
Xtr, Xte, ytr, yte = train_test_split(df[cot8], df["gia_nha"], test_size=0.2,
random_state=42)

cac_mo_hinh = {
    "LinReg bậc 1": make_pipeline(StandardScaler(), LinearRegression()),
    "LinReg bậc 3": make_pipeline(PolynomialFeatures(3), StandardScaler(),
LinearRegression()),
```

```
    "Ridge bậc 3":  make_pipeline(PolynomialFeatures(3), StandardScaler(),
Ridge(alpha=10)),
    "Lasso bậc 3":  make_pipeline(PolynomialFeatures(3), StandardScaler(),
Lasso(alpha=0.5, max_iter=50000)),
}
for ten, mh in cac_mo_hinh.items():
    mh.fit(Xtr, ytr)
    w = mh[-1].coef_
    print(f"{ten:<13}: số đặc trưng = {len(w):>3} | khác 0 = {(np.abs(w) > 1e-
6).sum():>3}"
          f" | R² train = {mh.score(Xtr, ytr):.3f} | R² test = {mh.score(Xte,
yte):.3f}")
```

Kết quả mẫu (Lasso có thể mất khoảng 1 phút):

```
LinReg bậc 1 : số đặc trưng =   8 | khác 0 =   8 | R² train = 0.595 | R² test =
0.587
LinReg bậc 3 : số đặc trưng = 165 | khác 0 = 164 | R² train = 0.742 | R² test =
-0.358
Ridge bậc 3  : số đặc trưng = 165 | khác 0 = 164 | R² train = 0.683 | R² test =
0.672
Lasso bậc 3  : số đặc trưng = 165 | khác 0 =  22 | R² train = 0.645 | R² test =
0.639
```

Một câu chuyện trọn vẹn trong 4 dòng:

1. Bậc 1: underfit (0,595 / 0,587).
2. Bậc 3 không regularization: **overfit nặng**. Train tăng lên 0,742, nhưng test **âm** (−0,358), tệ hơn cả đoán trung bình! (Vài khu có giá trị đặc trưng cực đoan khiến đa thức bậc 3 "nổ".)
3. Ridge bậc 3: R^2 test **0,672**, tốt hơn hẳn mô hình ban đầu (0,587).
4. Lasso bậc 3: chỉ giữ **22 / 165** đặc trưng mà vẫn đạt 0,639. Gọn hơn nhiều, dễ giải thích hơn.

(Cột "khác 0 = 164" chứ không phải 165 vì cột hằng số "1" do PolynomialFeatures tạo ra bị StandardScaler biến thành 0.)

---

8. Gợi ý trả lời phỏng vấn (mục 1)

Câu 1 (train 0,99, test 0,40): "Đây là overfitting: mô hình học thuộc cả nhiễu của tập train. Em sẽ: (1) kiểm tra lại cách chia, xem có rò rỉ dữ liệu không; (2) thêm dữ liệu nếu có thể; (3) giảm độ phức tạp mô hình hoặc bớt đặc trưng; (4) thêm regularization như Ridge/Lasso, chọn alpha bằng cross-validation."

Câu 2 (cả hai 0,55): "Đó là underfitting: mô hình quá đơn giản. Em sẽ thêm đặc trưng tốt hơn (feature engineering), thử mô hình phức tạp hơn như đa thức có regularization hoặc mô hình dạng cây, và giảm regularization nếu đang dùng."

Câu 3 (Ridge vs Lasso): "Ridge phạt tổng bình phương hệ số, làm hệ số nhỏ lại nhưng hiếm khi bằng 0, và ổn định khi các đặc trưng tương quan với nhau. Lasso phạt tổng trị tuyệt đối, đẩy nhiều hệ số về đúng 0 nên tự

chọn đặc trưng. Em chọn Lasso khi nghi có nhiều đặc trưng vô ích hoặc cần mô hình gọn, dễ giải thích. Cả hai đều cần chuẩn hóa dữ liệu trước."

---

9. Bài tập

1. Trong thí nghiệm xe máy (mục 2.3), thử thêm bậc 5 và bậc 9. Vẽ biểu đồ lỗi train và lỗi test theo bậc 1, 3, 5, 9, 15. Hình dạng có giống sơ đồ ở mục 3.4 không?
2. **Tính tay** loss Lasso cho hai mô hình ở mục 5.2 (A: w = [10, −8]; B: w = [1; 0,5]; cùng MSE = 2; α = 0,1).
3. Trong mục 7, thử Ridge với α = 1, 100, 1000. Ghi R^2 train và test. α nào tốt nhất?
4. In ra 22 đặc trưng mà Lasso giữ lại ở mục 7. (Gợi ý: `mh[0].get_feature_names_out(cot8)` cho tên các cột đa thức.) Đặc trưng nào có hệ số lớn nhất?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. Lỗi train giảm dần đều khi bậc tăng. Lỗi test giảm rồi **tăng vọt** ở bậc cao: đúng hình chữ U như sơ đồ.

```
bac_list = [1, 3, 5, 9, 15]; loi_tr, loi_te = [], []
for bac in bac_list:
    mh = make_pipeline(PolynomialFeatures(bac), StandardScaler(),
LinearRegression()).fit(X_train, y_train)
    loi_tr.append(root_mean_squared_error(y_train, mh.predict(X_train)))
    loi_te.append(root_mean_squared_error(y_test, mh.predict(X_test)))
plt.plot(bac_list, loi_tr, "o-", label="Train"); plt.plot(bac_list, loi_te, "o-",
label="Test")
plt.yscale("log"); plt.xlabel("Bậc đa thức"); plt.ylabel("RMSE"); plt.legend();
plt.show()
```

2. A: 2 + 0,1 × (10 + 8) = **3,8**. B: 2 + 0,1 × (1 + 0,5) = **2,15**. Lasso cũng chọn B.
3. Kết quả thật (R^2 train / test): α = 1 → 0,706 / **0,686**; α = 10 → 0,683 / 0,672; α = 100 → 0,664 / 0,658; α = 1000 → 0,640 / 0,632. Trong các giá trị đã thử, **α = 1** tốt nhất. α càng lớn, cả train và test cùng giảm: đang trượt dần về phía underfit. (Cách chọn α đúng chuẩn bằng cross-validation, không nhìn tập test: bài 2.18.)
4. Code:

```
lasso_mh = cac_mo_hinh["Lasso bậc 3"]
ten_cot = lasso_mh[0].get_feature_names_out(cot8)        # tên 165 cột đa thức
he_so = pd.Series(lasso_mh[-1].coef_, index=ten_cot)
print(he_so[he_so.abs() > 1e-6].sort_values(key=abs, ascending=False).round(2))
```

Kết quả thật, 4 hệ số lớn nhất: `vi_do` (−62,0), `thu_nhap kinh_do^2` (58,6), `ti_le_phong_ngu nguoi_moi_ho^2` (51,1), `kinh_do` (−46,8). Vị trí và thu nhập vẫn chiếm ưu thế, khớp với EDA ở bài 1.12. Nhưng để ý các tích kỳ lạ như "tỉ lệ phòng ngủ × (người mỗi hộ)²": **khó giải thích** bằng lời. Đó là cái giá của đặc trưng đa thức: chính xác hơn, nhưng mất tính dễ hiểu của Linear Regression.

---

10. Flashcard

Mặt trước	Mặt sau
Underfitting nhận biết thế nào?	Lỗi train và test đều cao, gần nhau.
Overfitting nhận biết thế nào?	Lỗi train thấp, lỗi test cao hơn nhiều.
Bias cao nghĩa là?	Mô hình quá cứng nhắc, sai hệ thống.
Variance cao nghĩa là?	Mô hình quá nhạy với dữ liệu train cụ thể.
Có nên chọn mô hình theo lỗi train?	Không. Lỗi train luôn giảm khi phức tạp hơn.
Cách chữa overfitting tốt nhất?	Thêm dữ liệu. Sau đó: regularization, đơn giản hóa mô hình.
Loss của Ridge?	MSE + $\alpha \cdot \Sigma w^2$
Loss của Lasso?	MSE + $\alpha \cdot \Sigma \lvert w \rvert$
Khác biệt chính Ridge vs Lasso?	Lasso đẩy hệ số về đúng 0 (tự chọn đặc trưng).
α lớn quá thì sao?	Underfitting.
Tham số C trong LogisticRegression?	C = 1/α. C nhỏ = phạt mạnh.
Đa cộng tuyến là gì, chữa bằng gì?	Các đặc trưng tương quan mạnh → hệ số không ổn định. Ridge.

Bài tiếp theo: 2.7 – KNN (K-Nearest Neighbors).